NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

ALEXANDRE MIGUEL DA SILVA PERES

BSc in Computer Science and Engineering

PLATFORM FOR DISSERTATION/PROJECT PROPOSAL AND MANAGEMENT

Dissertation Plan
MASTER IN COMPUTER SCIENCE AND ENGINEERING
SPECIALIZATION IN PROGRAMMING LANGUAGES AND SOFTWARE SYSTEMS

NOVA University Lisbon
Draft: *February 6, 2026*

PLATFORM FOR DISSERTATION/PROJECT PROPOSAL AND MANAGEMENT

ALEXANDRE MIGUEL DA SILVA PERES

BSc in Computer Science and Engineering

Adviser: Nuno Preguiça
Full Professor, NOVA University Lisbon

Co-adviser: Vítor Duarte
Assistant Professor, NOVA University Lisbon

Dissertation Plan
MASTER IN COMPUTER SCIENCE AND ENGINEERING
SPECIALIZATION IN PROGRAMMING LANGUAGES AND SOFTWARE SYSTEMS

NOVA University Lisbon
Draft: *February 6, 2026*

Abstract

The management of internships, dissertations, and engineering projects at the Department of Computer Science (DI) of NOVA School of Science and Technology (FCT) currently relies on a legacy platform that is technically obsolete and limited to undergraduate cycles. Master's degree processes are supported by spreadsheets and informal email exchanges, leading to fragmented data, audit difficulties, and high administrative burden for coordinators, professors, companies, and the secretariat. Simultaneously, the existing platform suffers from UX problems, email deliverability failures, and security vulnerabilities, making incremental evolution unfeasible.

This problem is relevant and challenging because internship placement is a critical process for students' academic paths and for the department's relationship with companies. The solution must reconcile complex functional requirements (automated seriation, multiple user profiles, distinct workflows for different degrees) with demanding non-functional requirements (security, auditability, interoperability, and usability) in an institutional context where long-term maintainability is as important as development speed.

This dissertation proposes the design and implementation of a new unified platform for managing proposals and candidacies, based on a layered architecture: a React frontend, a backend initially supported by Supabase and subsequently evolved to Spring Boot, and a PostgreSQL database. The work follows an iterative methodology: requirements gathering, process (BPMN) and data (ERD) modeling, use case definition, and accelerated prototyping through Vibe Coding tools, followed by refactoring and architectural hardening.

As main results, a functional prototype is expected to automate critical tasks (seriation, notifications, protocols), improve user experience and data quality, and provide a critical analysis of the user of Vibe Coding tools in institutional contexts.

Keywords: Internship and dissertation management · Web Platforms · Software Architectures · Vibe Coding · Full-Stack Development

Resumo

A gestão de estágios, dissertações e projetos de engenhharia no Departamento de Informática da NOVA FCT depende atualmente atualmente de uma plataform legada, tecnicamente obsoleta e limitada aos ciclos de licenciatura. Os processos do mestrado são suportados por folhas de cálculo e trocas de e-mail informais, originando dados fragmentados, dificuldades de auditoria e uma carga administrativa elevada para coordandores, professores, empresas e secretaria. Em paralelo, a plataforma existente sofre de problems de UX, falhas de entregabilidade de emails e fragilidades de segurança, tornando inviável a sua evolução incremental.

Este problema é relevante e desafiante porque a atribuição de estágios é um processo crítico para o percuso académico dos estudantes e para a relação do departamentto com as empresas. A solução tem de conciliar requisitos funcionais complexos (seriação automática, múltiplos perfis e utilizador, fluxos distintos para licenciatura) com requisitos não funcionais exigentes (segurança, auditabilidade, interoperabilidade, e usabilidade), num contexto institucional onde a manutenção a longo prazo é tão importante como a rapidez de desenvolvimento.

Esta dissertação propõe o desenho e a implementação de uma nova plataforma web unificada para gestão de propostas e candidaturas, assente numa arquitetura em camadas: frontend React/Next.js, backend inicialmente suportado por Supabase e posteriormente evoluído para Spring Boot, e base de dados PostgreSQL. O trabalho segue uma metodologia iterativa: levantamento requisitos, modelação de processos (BPMN) e dados (ERD), definição de casos de uso e prototipagem acelerada através de ferramentas de Vibe Coding, seguida de refactoring e endurecimento arquitetural.

Como principais resultados, espera-se um protótipo funcional que automatiza tarefas critícas (seriação, notificações, protocolos), melhor a experiencia de utilização e a qualidade dos dados, e uma análise crítica do uso de ferramentas de Vibe Coding em contexto institucionais

Palavras-chave: Gestão de estágios e dissertações · Plataformas Web · Arquiteturas de Software · Vibe Coding · Desenvolvimento Full-Stack

Contents

Acronyms **xi**

1 Introduction **1**
1.1 Context and Background . . . 1
1.2 Problem Statement . . . 1
1.3 Objectives . . . 3
1.4 Methodology . . . 3

2 State of the art and Technologies **7**
2.1 Backend Architectures and BaaS . . . 7
2.1.1 Supabase (Backend-as-a-Service) . . . 8
2.1.2 Spring Boot (Custom Backend) . . . 8
2.1.3 Node.js and Express (Custom Backend) . . . 9
2.2 Data Persistence: SQL vs. NoSQL . . . 9
2.2.1 The Relational Model (SQL) . . . 10
2.2.2 The Non-Relational Model (NoSQL) . . . 11
2.3 Frontend Frameworks and User Experience . . . 12
2.3.1 React.js . . . 12
2.3.2 Angular . . . 13
2.4 AI-Assisted Development *(Vibe Coding)* . . . 14
2.4.1 Analysis of Vibe Coding Tools . . . 14
2.5 Summary and Justification of Technological Choices . . . 16
2.5.1 Backend Architecture . . . 16
2.5.2 Data Persistence . . . 17
2.5.3 Frontend Frameworks and Styling . . . 17
2.5.4 AI-Assisted Development (*Vibe Coding*) . . . 18

3 Requirements and Current System Analysis **19**
3.1 Analysis of the Legacy System . . . 19
3.1.1 Infrastructure and Tech Debt . . . 19

3.1.2 Functionalities and Limitations per Role 20
3.1.3 Data and Reports 20
3.2 Requirements Gathering 21
3.2.1 Elicitation Methods 21
3.2.2 Stakeholders 22
3.2.3 Functional Requirements 22
3.2.4 Non-Functional Requirements 23
3.3 Use Cases 24
3.3.1 Actors and Their Goals 24
3.3.2 High-Level Use Case Diagram 25
3.3.3 Detailed Use Case Specifications 26
3.3.4 Use Case Relationships and Extensions 27
3.3.5 Summary of Use Cases 27

4 Solution Proposal **29**
4.1 Proposed System Architecture 29
4.2 Process Modelling (BPMN) 30
4.3 Data Model (ERD) 30
4.4 Work Plan 30

Bibliography **31**

Annexes

I Annex 1 - Functional and Non-Functional Requirements **35**

II Annex 2 - Use Cases Specifications **43**

List of Figures

3.1 High-Level Use Case Diagram of the system with core functionalities 25

List of Tables

2.1 Comparison of Backend Frameworks and Platforms 9
2.2 Comparison of Data Persistence Models 12
2.3 Comparison of Frontend Technologies 14
2.4 Comparison of Vibe Coding Tools 16

I.1 Functional Requirements - Edition and Configuration Management 35
I.2 Functional Requirements - Proposal Submission and Classification 36
I.3 Functional Requirements - Student Candidacies and Seriation 36
I.4 Functional Requirements - Company and User Management 37
I.5 Functional Requirements - Communication and Notifications 37
I.6 Functional Requirements - Reporting and Data Export 38
I.7 Functional Requirements - Master's Dissertation Lifecycle Support 38
I.8 Non-Functional Requirements - Security 39
I.9 Non-Functional Requirements - Auditability & Compliance 39
I.10 Non-Functional Requirements - Reliability & Availability 39
I.11 Non-Functional Requirements - Performance & Scalability 40
I.12 Non-Functional Requirements - Usability & Accessibility 40
I.13 Non-Functional Requirements - Interoperability & Data Exchange 40
I.14 Non-Functional Requirements - Maintainability & Extensibility 41
I.15 Non-Functional Requirements - Email Deliverability (Critical Legacy Fix) . . 41

II.1 Use Case Specification: Proposal Ranking 44
II.2 Use Case Specification: Coordinator Validates and Provides Feedback on Proposal 45
II.3 Use Case Specification: System Executes Student Seriation Algorithm 47
II.4 Use Case Specification: Company/Professor Responds to Coordinator Feedback and Resubmits Proposal 49
II.5 Use Case Specification: Edition Coordinator Creates New Edition 51
II.6 Use Case Specification: Company Representative Registers and Submits Initial Proposal 53

II.7 Use Case Specification: Student Submits Support Documentation 55
II.8 Use Case Specification: Company/Professor Views Assigned Candidates List and Schedules Interviews 57
II.9 Use Case Specification: Secretariat Verifies Payment and Finalizes Protocol . 58
II.10 Use Case Specification: Edition Coordinator Publishes Call for Proposals . . 60
II.11 Use Case Specification: Administrator Performs Superuser Access for Student Support 61

Acronyms

ACID Atomicity, Consistency, Isolation, Durability *(pp. 10, 11, 17)*
AI Artificial Intelligence *(pp. 4, 7, 12, 13, 16)*
API application programming interface *(pp. 7–9, 16, 17, 24)*

BaaS Backend-as-a-Service *(pp. v, 4, 7, 8, 16)*
BASE Basically Available, Soft State, Eventually Consistent *(pp. 10, 11)*
BPMN Business Process Model and Notation *(p. 4)*

CRUD Create, Read, Update, Delete *(pp. 8, 18)*
CSV Comma-Separated Values *(p. 20)*

DI Department of Computer Science *(pp. 1, 3, 7, 10, 19)*

EJS Embedded JavaScript *(p. 9)*
ERD Entitiy-Relationships Diagrams *(p. 4)*

FCT NOVA School of Science and Technology *(pp. 1, 3, 7, 19)*

HTML HyperText Markup Language *(p. 9)*
HTTP Hypertext Transfer Protocol *(p. 19)*
HTTPS Hypertext Transfer Protocol Secure *(p. 23)*

I/O Input/Output *(p. 9)*
IDE integrated development environment *(p. 14)*
ISR Incremental Static Regeneration *(p. 18)*

JPA Java Persistence API *(p. 8)*
JSON JavaScript Object Notation *(pp. 10, 17, 20, 29)*
JSONB JavaScript Object Notation Binary *(pp. 10, 17, 29)*

JSX JavaScriptXML *(p. 12)*

LLM large language model *(pp. 7, 14)*

MVP Minimum Viable Product *(p. 15)*

NoSQL Not only SQL *(pp. v, 4, 9–11, 17)*
NOVA NOVA University Lisbon *(pp. 1, 3, 7, 19)*

RBAC role-based access control *(p. 23)*
RLS Row-Level-Security *(pp. 8, 17, 23, 29)*

SPA Single Page Application *(p. 12)*
SQL Structured Query Language *(pp. v, 4, 8–11, 17)*
SSG Static Site Generation *(pp. 13, 18)*
SSL Secure Sockets Layer *(p. 19)*
SSR Server-Side-Rendering *(pp. 13, 18)*

TLS Transport Layer Security *(pp. 19, 23)*

UI User Interface *(pp. 12, 14, 17, 18, 20)*
UX User Experience *(pp. 2, 4, 12, 15)*

WYSIWYG What You See Is What You Get *(p. 20)*

1

Introduction

1.1 Context and Background

The transition of students from academic life to the professional market or advanced research is a key part of the university experience. This is mainly supported through three different distinct paths at the Department of Computer Science (DI) of NOVA School of Science and Technology (FCT):

- **Undergraduate Internships**: Primarily offered by external companies to provide students with their first professional experience.
- **Master's Dissertations**: Scientific thesis typically conducted within the university, focusing on academic research under the supervision of professors.
- **Engineering Projects**: Practical projects proposed by external companies or the Department of Informatics itself, allowing Master's students to apply advanced engineering principles to real-world projects.

Managing those processes involves a complex coordination of multiple stakeholders: students seeking opportunities aligned with their skills and interests, professors coordinating editions, supervising students and also proposing dissertation themes, and external companies providing practical challenges. Within this ecosystem, a reliable digital platform is essential to act as a central hub for communication, proposal submission, and student placement.

1.2 Problem Statement

At DI at NOVA FCT, the management of internships, dissertations and engineering projects involves a multi-stage lifecycle that includes proposal submission, clarification and approval processes, making proposals available to students, assigning projects to students, report/dissertation submission, and providing documentation to juries. Currently, the department relies on a legacy platform to support some of these activities, however this

system presents several critical issues that hinder the efficiency of the department.
The most significant limitation is that the current platform is not being utilized for the Master's degree processes - including both scientific dissertations and engineering projects. At the moment it is exclusively used to manage undergraduate internships, meaning that the more complex workflows associated with Master's students are handled through manual, or non-standardized methods. This creates a lack of centralized data an increases the administrative burden on coordinators, professors and the secretariat.
Furthermore, even within its current scope, the legacy system suffers from several functional and technological gaps:

- **Communication Gaps and Reliability**: The current method for "*Call for Proposals*" is manual, requiring coordinators to send individual emails to companies, advertising the opportunity send proposals, for each edition. There are no safeguards to ensure that these emails follow specific deliverability rules, often resulting in messages being flagged as spam. Additionally, the system lacks the flexibility to create and edit email templates, preventing coordinators from customizing communication for different editions or context.

- **Inefficient Feedback Loops**: Although companies can submit proposals, there is no integrated mechanism for coordinators to provide direct feedback on those proposals for clarification. This forces corrections to be handled outside the platform, whereas a centralized system would allow companies to correct and resubmit proposals based on coordinator comments.

- **Stale Company Data**: The existing repository contains a large amount of outdated information, including companies that no longer exist or have changed their contacts or other information. There is a lack of a "validation" status to ensure only active and verified companies can send proposals.

- **Manual Protocol Management**: For new companies that have never participated in previous editions, the process of signing the required protocols is still entirely manual. The platform should automate delivery and tracking of these legal documents as soon as a new company registers.

- **Outdated Technology and User Experience (UX)**: The platform is built on an obsolete technology stack that is hard to maintain, difficult to update, and exposes security risks. At the same time, the user interface is not intuitive for external stakeholders, making it difficult for companies to register, manage employee accounts, or monitor the status of their proposals in real time.

In conclusion, the necessity for a new platform arises from the need to unify all academic degrees under a single digital infrastructure, replacing manual tracking with a secure, automated system that ensures data integrity and seamless experience for all stakeholders.

1.3 Objectives

The primary objective of this dissertation is to design and develop a modernized web-based platform that unifies and automates the management and entire lifecycle of undergraduate internships, Master's dissertations, and engineering projects for DI at NOVA FCT. The projects aim to transition from a fragmented, manual system to a centralized digital ecosystem.
To achieve this goal, the following specific objectives have been defined for the initial version:

- **Process Unification**: Integrate the lifecycles of undergraduate internships and Master's degree pathways (both scientific and engineering-focused) into a single platform, replacing the current manual methods.
- **Workflow Automation**: Implement automated procedures for the "*Call for Proposals*", the validation of proposals, the student *seriation* (ranking) and the assignment of students to a proposal.
- **Enhanced Communication and Feedback**: Establish direct feedback loops within the platform, allowing coordinators to request clarifications on proposals and ensuring companies can resubmit corrected versions.
- **Document and Protocol Management**: Automate the delivery, tracking, and storage of legal protocols, student CVs, and recommendation letters.
- **Administrative Oversight**: Develop advanced administrative tools, including "*superuser*" access for student support, detailed system logs for auditing, and validation workflow for new company registrations.

1.4 Methodology

This project will follow an Agile/Scrum methodology, characterized by iterative development, frequent feedback loops [33] to ensure the final platform aligns with the university's complex requirements. The work plan is structured into several phases, prioritizing rapid prototyping followed by a transition to a high-performance custom architecture.
To achieve this goal, the following methodological steps will be used for the design, implementation, and validation of the system:

1. **Requirement Gathering and Analysis**: A thorough analysis of the legacy system and current manual processes was conducted to identify functional and non-functional requirements. This phase includes the definition of user roles (*Admin*, *Coordinator*, *Student*, *Professor*, *Company*, *Secretariat*) and their respective use cases.

2. **System Modeling and Design**: Utilizing Business Process Model and Notation (BPMN) to map complex workflows [25] and Entitiy-Relationships Diagrams (ERD) to design the database schema [7]. This phase also involves planning the system architecture.

3. **AI-Assisted Rapid Prototyping (*Vibe Coding*)**: The initial implementation phase will leverage *Vibe Coding* tools, such as Lovable, Figma Make or Google AI Studio, to quickly generate the frontend and core functional interfaces [9]. These tools excel at creating user interfaces through iterative natural language prompting. During this stage, Supabase will be utilized as Backend-as-a-Service (BaaS) to provide an immediate infrastructure for authentication, database management, and initial data structures via SQL[40]. The prototype produced in this phase will serve as the baseline that will be iteratively improved and hardened. This approach allows for the rapid fulfillment of most functional requirements while maintaining a live, testable version of the platform.

4. **Progressive Refinement and Continuous Development**: Once the prototype reaches a stable state and meets the core functional requirements, the focus shifts from "*getting this working*" to improving code quality, structure, and user experience. In this phase the existing codebase generated by AI is refactored, extended, and optimized manually and potentially still with the help of AI-assisted tools. The goal is to evolve the prototype into a maintainable system, improving UX, performance, and adherence to architectural guidelines.

5. **Custom Backend Evolution**: To support more complex business logic - such as specialized student *seriation* (ranking) algorithms and robust administrative auditing - the system progressively transitions towards a dedicated custom backend, likely using Spring Boot [35]. While Supabase remains valuable for rapid development, a Spring Boot-based architecture provides stronger domain modelling, type safety and maintainability through its layered structure (Controller, Service, Repository). This evolution builds on top of the existing data model and functionality, ensuring that the platform remains usable while gaining scalability and reliability for long-term university use.

6. **Verification and Validation**: The platform will be tested against the initial requirements, ensuring that automated emails follow specific deliverability rules avoiding being flagged as spam. Additionally, the seriation logic will be rigorously validated to ensure it correctly handles student placements and matching according to pre-defined criteria.

7. **Documentation and Evaluation**: There will be a continuous documentation of the architecture and design decisions, providing detailed justifications for technical choices such as the preference for SQL over NoSQL for structured academic data,

and other technical choices. At the end, the platform and development process will be tested and evaluated with respect to the initial goals, highlighting limitations and possible directions for future work.

2

State of the art and Technologies

This chapter examines the current state of software engineering practices and technologies relevant to building a modern platform for the DI at NOVA FCT. The goal is to overcome limitations of the legacy system - such as fragmented data, manual workflows, and inefficient communication - by evaluating contemporary approaches to backend architectures, data persistence, frontend frameworks, and AI-assisted development workflows. Particular attention is given to the emerging paradigm of AI-assisted software development, often described as "*Vibe Coding*" [14] which supports rapid prototyping by allowing developers to steer large language models (LLMs) through natural-language interactions instead of traditional line-by-line coding.

Modern web application stacks are characterized by a wide range of choices at each layer, from infrastructure and backend services to frontend frameworks and development workflows. On the backend, developers must choose between fully managed BaaS platforms [16] and custom backends that offer fine-grained control over domain logic, performance, and governance. On the frontend, single-page web applications and component-based frameworks (such as React, Angular, or Vue) are commonly combined with RESTful or GraphQL APIs, enabling decoupled client-server architectures and iterative delivery of new features.

While "*Vibe Coding*" and other AI-assisted practices promise faster experimentation and reduced boilerplate, they also introduce new trade-offs regarding maintainability, security, and auditability in production systems [32].

The remainder of this chapter critically reviews these technologies and architecture choices in light of the requirements identified for the new platform.

2.1 Backend Architectures and BaaS

The choice of backend architecture directly influences development speed, scalability, operational complexity, and the ability to encode complex business rules such as student seriation and multirole access control. For this project, two main paradigms are considered: Backend-as-a-Service solutions, which offer ready-made infrastructure and managed

services, and custom backends built with mature frameworks such as Spring Boot and Express.js, which provide full control over domain modeling, integration patterns, and deployment environments. [5]

BaaS platforms typically optimize for rapid time-to-market by abstracting away server provisioning, database management, and authentication, making them well-suited for early-stage prototyping and small teams [1]. In contrast, custom backends allow tailoring of data models, ownerships of the entire codebase, and fine-grained observability, which becomes critical for institutional applications that must satisfy compliance, long-term maintainability, and integration with existing academic processes.

2.1.1 Supabase (Backend-as-a-Service)

Supabase is an open-source BaaS stack that builds on PostgreSQL and exposes automatically generated REST APIs through PostgREST [38], significantly reducing boilerplate CRUD implementation effort. By providing an integrated suite of tools - including database, authentication, storage, and real-time capabilities - Supabase enables rapid development cycles while leveraging a relational data model that is compatible with established SQL tooling and practices. [40]

A central feature of Supabase is its built-in authentication system, which supports user registration, password-based login, magic links, and role-based access control[36], simplifying the enforcement of different permission profiles for students, companies and professors. Row-Level-Security (RLS) policies can be defined directly at the database level, ensuring that authorization rules are enforced close to the data and cannot be bypassed by misconfigured API clients [39]. For logic that goes beyond standard CRUD operations - such as orchestrating notification workflows or managing student seriation - Supabase offers serverless "Edge Functions", typically written in TypeScript or JavaScript, which run in a managed environment near end users to minimize latency. [37]

2.1.2 Spring Boot (Custom Backend)

Spring Boot is a widely adopted framework for building production-grade Java or Kotlin backends and is especially suitable for complex, domain-driven systems that require strict typing, layered architectures, and extensive integration capabilities. Its opinionated configuration model and extensive ecosystem (Spring Data, Spring Security, Spring Cloud) allows developers to structure applications into controllers, services, and repositories, facilitating clear separation of concerns and maintainable domain models. [35]

For academic workflows that must support student ranking algorithms, complex eligibility rules, and auditable decision-making processes, the type safety and testability provided by Spring Boot are particularly advantageous. The framework integrates with JPA and tools like Hibernate Envers to support automatic auditing of entity changes [35, 6], which helps track user actions, state transitions, and login-related events - capabilities

that are often required for administrative oversight and internal compliance in university environments.

2.1.3 Node.js and Express (Custom Backend)

Express.js is a minimalist web framework for Node.js that offers a lightweight and flexible foundation for building HTTP APIs using JavaScript or TypeScript [13]. Its event-driven, non-blocking I/O model allows a single process to handle many concurrent connections efficiently, making it well-suited for notification-heavy workloads [24].

The Node.js ecosystem provides a rich set of libraries for templated email generation and delivery, including tools such as Handlebars or EJS to render dynamic HTML email bodies [17, 12] for coordinators and administrators. Combined with tasks schedulers or message queues, Express-based backends can orchestrate asynchronous communications and background jobs [24], helping address a key functional gap of legacy systems that rely on manual email workflows and ad hoc templates.

Summary

In summary, Supabase offers a managed, high-velocity environment that is ideal for rapid prototyping, while Spring Boot and Express.js provide greater control over domain logic and integration at the cost of more boilerplate and operational responsibility. As shown in Table 2.1, this project adopts Supabase in the early stages and Spring Boot as the long-term backbone for institutional deployment.

Feature	**Supabase (BaaS)**	**Spring Boot**	**Node.js (Express)**
Architectural Pattern	Managed Infrastructure	Layered / Monolithic	Middleware-based
Development Velocity	Accelerated (Managed)	Moderate (Boilerplate)	High (Minimalist)
Domain Control	Ecosystem-dependent	High (Type-safe)	High (Flexible)
Integrations	Webhooks & Extensions	Extensive / Enterprise	Package-rich (NPM)
Security Model	Row-Level Security (RLS)	Spring Security (OIDC/JWT)	Custom Middleware

Table 2.1: Comparison of Backend Frameworks and Platforms

2.2 Data Persistence: SQL vs. NoSQL

The selection of a data persistence model is a fundamental architectural decision that

dictates how the system ensures data integrity, scalability, and retrieval efficiency [43, 4]. Given the complex ecosystem of the thesis and internships management, which involves multiple stakeholders (Students, Professors, Companies, Secretariat, etc...) and multi-stage lifecycle workflows for academic projects, this section evaluates the trade-offs between Relational (SQL) and Non-Relational (NoSQL) databases.

The choice between SQL and NoSQL has complex implications for institutional systems, where relational databases typically prioritize data integrity and consistency through Atomicity, Consistency, Isolation, Durability (ACID) compliance [29], while non-relation systems often follow the Basically Available, Soft State, Eventually Consistent (BASE) model, trading immediate consistency for horizontal scalability and schema flexibility [42].

2.2.1 The Relational Model (SQL)

Relational databases structure data into tables with predefined schemas, utilizing Foreign Keys to establish strict relationships between entities [18]. This approach provides fundamental guarantees about data quality and consistency that are essential for institutional systems.

- Structured Data and Integrity: The academic context requires rigorous data consistency. For instance, a Candidacy must strictly link to an existing Student and a valid Proposal. The relational model enforces referential integrity through primary and foreign key constraints, preventing "orphaned" records - a situation where a child record references a non-existent parent record [34]. This is particularly critical in the DI management platform, where deleted Companies should not retain active Proposals, and Students cannot be assigned to non-existent projects.

- Referential integrity mechanism ensure that every foreign key in a child table corresponds to a valid primary key in the parent table, maintaining logical consistency across the entire database. Beyond preventing data anomalies, referential integrity supports cascading updates and deletions, ensuring that changes propagate correctly across related entities [18, 34].

- Complex Querying: The platform requires complex join operations to generate views - for example, listing all proposals for a specific edition filtered by a company's validation status, or retrieving all students that are candidates to a particular internship. SQL excels at these complex relationships through its mature and optimized query language, which can efficiently traverse multiple table relationships in a single query [21].

- PostgreSQL: This project will initially utilize PostgreSQL (via Supabase for rapid prototyping). Beyond standard SQL features, PostgreSQL offers support for complex data types (JSON/JSONB, arrays, enums) and Stored Procedures [26, 27]. This allows

the system to handle semi-structured data - such as like flexible feedback forms or email template configurations - while maintaining the benefits of a relational schema for academic entities.

2.2.2 The Non-Relational Model (NoSQL)

NoSQL databases (e.g., document stores like MongoDB or key-value systems like Redis) offer schema flexibility and horizontal scalability, often at the cost of immediate consistency (ACID properties) [43]

- Flexibility vs Structure: While NoSQL allows for rapid iteration of data models without schema migrations, the academic domain is highly structured. Entities such as Users, Editions, Proposals, Companies and Students have well-defined attributes that rarely change dynamically or unexpectedly. The governance requirements of an academic institution demand clear, predictable data structures rather than evolving, loosely-typed records.

- Horizontal Scalability: A key advantage of many NoSQL systems is the ability to easily scale horizontally across multiple nodes, distributing data and load to handle very high traffic volumes or massive datasets. This is attractive for large, globally distributed consumer applications, but less critical for an institutional platform whose workload is bounded by the number of students, companies and editions. In this context, the added operational complexity of managing a distributed NoSQL cluster is not justified by the expected scale, especially when a single relational database can already handle the anticipated load with proper indexing and tuning.

- Consistency Challenges: In workflows like Student Seriation, data consistency is essential. A NoSQL approach, which often relies on eventual consistency, could introduce race conditions where a student is assigned to a project that is no longer available or where multiple simultaneous requests lead to conflicting state changes. The BASE model prioritizes availability and partition tolerance over strict consistency, which is acceptable for read-heavy analytics systems but unsuitable for transactional workloads that require immediate correctness.

Summary

For the internship and dissertation management platform, the structured nature of the domain, strict consistency requirements, and moderate scale favor a relational SQL database over a NoSQL solution. Table 2.2.2 summarizes the main differences between relational and non-relational models in terms of schema, consistency, integrity, querying, and institutional fit, and it motivates the choice of PostgreSQL as the primary data store for this project.

Criterion	**Relational (SQL)**	**Non-Relational (NoSQL)**
Schema Definition	Rigid/Predefined	Schema-less/Dynamic
Consistency Model	ACID Compliance	BASE (Eventual)
Data Integrity	Enforced (Foreign Keys)	Application-level logic
Query Complexity	Native Relational Joins	Aggregation Pipelines
Scaling Strategy	Vertical Scaling	Horizontal Partitioning
Institutional Fit	High (Structured Records)	Low (Consistency Risks)

Table 2.2: Comparison of Data Persistence Models

2.3 Frontend Frameworks and User Experience

The "Outdated User Experience (UX)" of the legacy system is identified as a critical failure point, particularly for external stakeholders such as companies who struggle to register, manage accounts, or monitor proposal status. To address these limitations, the new platform must transition from static, server-rendered pages to a modern, reacting Single Page Application (SPA) or hybrid architecture that enables faster navigation, richer interactivity [30], and consistent interfaces across distinct user roles.

2.3.1 React.js

React is a free, open-source JavaScript library maintained by Meta that specializes in building interactive user interfaces through component-based architecture [30, 31]. Unlike traditional web development where the entire page must be reloaded on data changes, React efficiently re-renders only the components that have actually changed, using its virtual DOM reconciliation process to minimize performance overhead [22]. React uses JavaScriptXML (JSX), a syntax extension that allows developers to write HTML-like code directly within JavaScript [30], making the UI logic more readable and maintainable.

React is selected as the core library for building the platform's user interface for two primary reasons:

1. Component-Based Architecture: React allows for the creation of reusable, self-contained UI components (e.g. "ProposalCards" where all the details from a proposal are, that can be used in multiple pages) [28]. This modular approach ensures consistency across the distinct interfaces required for Students, Professors, and Companies, while enabling efficient code reuse and simplified maintenance.

2. Synergy with AI-Assisted Development: The project methodology relies on *Vibe Coding* tools for rapid prototyping. Tools such as Lovable, Bolt.new, Figma Make and Google AI Studio are optimized to generate code specifically for the React

ecosystem. Using React ensures that the code generated during the prototyping phase is immediately usable and extensible, eliminating the need for time-consuming library migrations.

In addition, React integrates naturally with full-stack frameworks like Next.js, which can provide Server-Side-Rendering (SSR), Static Site Generation (SSG), and routing natively when needed, without changing the underlying component model [23, 3, 20]. This means that if later iterations require improved performance for specific pages, the existing React components can be hosted within a Next.js application with minimal refactoring, preserving the benefits of the initial AI-assisted prototyping workflow.

2.3.2 Angular

Angular is a full-featured, opinionated frontend framework maintained by Google that provides an integrated solution for components, routing, forms, and state management in a single toolkit. Unlike React, which focuses on the view layer and relies on additional libraries for concerns such as routing, Angular ships with a more prescriptive structure (modules, services, dependency injection), which can be advantageous for large teams that value strong convetions.

Angular was considered as a viable alternative because if offers:

- Strong typing when used with TypeScript by default
- Built-in patterns for modularization and dependency injection
- Mature tooling for forms, validation, and reactive programming

Summary

In summary, the platform adopts a React-based frontend because it combines flexible component model, strong tool support, and excellent compatibility with AI-assisted "*Vibe Coding*" workflows, while still allowing future adoption of frameworks like Next.js for SSR or SSG if needed. Angular remains a robust alternative for institutional systems, but in this context its more opinionated structure and weaker alignment with current *Vibe Coding* tools make it less suitable, as reflected in the comparison of frontend technologies in Table 2.3. In the next section, the discussion of AI-assisted development will also show that React is the predominant target stack of the *Vibe Coding* tools considered for this project, further reinforcing its choice.

Feature	**React.js**	**Angular**
Core Paradigm	Component-based Library	Opinionated Framework
State Management	External (Redux)	Built-in (Services/RxJS)
Rendering Strategy	Client-Side (CSR) or Hybrid (SSR, SSG) when using Next.js	Client-Side (CSR)
Typed Support	Optional (TypeScript)	Native (TypeScript)
Prototyping Synergy	High (Vibe Coding tools)	Moderate

Table 2.3: Comparison of Frontend Technologies

2.4 AI-Assisted Development *(Vibe Coding)*

Traditional software development involves writing code manually, line by line, or using code assistants that give fragments of code in the developer's IDE, such as GitHub Copilot, Claude, or Gemini-based CLI tools. *Vibe Coding* tools differ from these assistants because instead of focusing on line-level code completion, they provide an integrated environment where a large language models (LLMs) generates and iteratively refines entire screens, workflows, or even full-stack applications through natural language instructions, with live preview and project-level context [8, 32]. This section analyzes the capabilities of such tools to determine their viability for the rapid prototyping phase of this project.

2.4.1 Analysis of Vibe Coding Tools

For this dissertation, there representative *Vibe Coding* tools were considered and explored: **Lovable, Bolt.new,** and **Figma Make**. They differ in scope (full-stack vs. frontend-centric), integration with existing workflows, and the amount of control they give developers over the generated code.

Lovable

Lovable focuses on providing an end-to-end environment for building web applications with minimal friction. From a single interface, it can generate React-based UIs, configure Supabase-backed database and backend, and deploy the resulting application. When requirements are underspecified, Lovable scaffolds the missing pieces on both frontend and backend, and offers automatic debugging and safety checks that make it forgiving for users with limited programming experience. A distinctive advantage is bidirectional GitHub synchronization: projects can be edited in an external IDEs and then synced back to Lovable. Although it started as a prototyping tool, its feature set and collaboration

capabilities make it a suitable for production-adjacent MVPs, internal tools and small applications [2].

Bolt.new

Bolt.new targets developers who want "prompt-to-full-stack" generation while retaining framework flexibility. It can scaffold applications across multiple stacks (React, Angular, Vue, React Native, etc.) and is backed by Supabase for backend and database support, similar to Lovable. Compared to Lovable, Bolt places more emphasis on exposing the underlying code and project structure, which provides greater control but also demands more debugging and configuration effort. It also supports working with real repositories and two-way GitHub sync. For teams that must target specific frameworks (for example, Angular or React Native), Bolt is a more natural fit than tools that assume React only, but its UX is less polished for non-experts [2].

Figma Make

Figma Make is tightly integrated into the Figma ecosystem and is therefore particularly attractive to designers. It starts from high-fidelity UI layouts and turns them into interactive prototypes that closely match the original design. This design-first approach is powerful for validating user flows and visual details with stakeholders. However, the generated code is harder to reuse outside Figma, and complexity grows quickly once custom logic and structure are layered on top. Although Figma Make supports backend integration (e.g., via Supabase), its current strength lies in rich prototypes rather than production-ready application code. For teams already working with Figma, its cost overhead is low, but they must be prepared to deal with frequent minor bugs and inconsistencies [2].

Summary

Vibe Coding tools such as Lovable, Bolt.new, and Figma Make complement traditional AI code assistants by operating at the level of full components and applications, which can dramatically reduce the time between a textual requirement and a testable interface, as summarized in Table 2.4. At the same time, their focus on speed means that generated code requires refactoring to fit a layered architecture, careful handling of security and vendor lock-in, and conventional testing before institutional deployment. For this project, these tools are therefore used as accelerators to bootstrap React-based prototypes that are later stabilized in a hand-curated codebase.

Criterion	Lovable	Bolt.new	Figma Make
Primary focus	End-to-end web apps	Full-stack code generation	Design-first interactive prototypes
Typical stack / output	React + Supabase	React, Angular, Vue, React Native, etc.	React + Supabase
Backend / DB support	Supabase backend & DB	Supabase backend & DB	Supabase backend & DB
Git / integration	Bi-directional GitHub sync	Bi-directional GitHub sync	One direction GitHub sync
Strengths	Complete environment, scaffolding, debugging, team-friendly	Framework flexibility, closer to "real" code	High visual fidelity, great for designers
Limitations	Mainly web/React; still evolving for large, complex domains	Rougher UX, more bugs and manual debugging	Generated code hard to reuse/maintain; more bugs; less suited to production

Table 2.4: Comparison of Vibe Coding Tools

2.5 Summary and Justification of Technological Choices

The preceding sections surveyed alternative technologies for the backend, data persistence layer, frontend frameworks, and AI-assisted development. This subsection summarizes and justifies the concrete choices adopted for the platform, in light of the requirements and constraints identified in Chapter 3.

2.5.1 Backend Architecture

Section 2.1 compared BaaS solutions with custom backend built with mature frameworks such as Spring Boot and Express.js. BaaS platforms like Supabase are particularly attractive for early-stage projects because they provide read-made infrastructure for authentication, storage, and real-time APIs, enabling rapid prototyping with small teams.

For this project, Supabase will be used in the initial phase to support AI-assisted, *Vibe Coding*-driven prototyping of the core workflows. However, the long-term target architecture is a custom backend implemented with Spring Boot. This choice is justified by:

- the need to code complex domain logic (student seriation rules, audit policies) with strong typing and explicit layering;
- the importance of testability and maintainability for an institutional system that is expected to evolve over multiple editions;
- the need to integrate cleanly with PostgreSQL features such as RLS and LISTEN/NOTIFY

In practice, the prototype built on Supabase is evolved rather than discarded: the data model and API contracts defined during prototyping are reused by the Spring Boot backend, minimizing migration effort.

2.5.2 Data Persistence

Section 2.2 contrasted relational (SQL) and non-relational (NoSQL) database models. Relational database prioritize integrity and consistency through ACID guarantees and referential constraints, while NoSQL systems offer schema flexibility and horizontal scalability at the cost of weaker consistency.

Given the structured nature of the problem domain-editions, students, companies, proposals, candidacies, and protocols - and the critical importance of correctness in operations such as seriation and assignments, a relational model is more appropriate.
The expected scale (bounded by the number of students and companies in the department) does not require the extreme horizontal scalability that motivates most NoSQL deployments.

PostgreSQL is therefore adopted as the primary data store because it:

- enforces referential integrity between core entities, preventing impossible states such as assignments to non-existent proposals or inactive companies ;
- supports RLS, which is essential for enforcing multi-tenant access rules directly at the database layer;
- provides JSON/JSONB columns for semi-structured data (*e.g.*, email templates, feedback metadata), allowing limited schema flexibility without introducing a separate NoSQL system.

2.5.3 Frontend Frameworks and Styling

Section 2.3 compared React and Angular as representative frontend technologies. In this project, React is chosen as the primary UI technology for two main reasons:

- its component-based model fits the need to offer multiple role-specific dashboards built from reusable components;

- it algins closely with the *Vibe Coding* tools used in the prototyping phase, which predominantly generate React Frontends

Next.js can be adopted around React when needed, main to provide routing and flexible rendering strategies (SSR, SSG, ISR) for pages that benefit from server-side or static rendering, without changing the underlying React component model. This combination supports both rapid prototyping and a clear path to production-ready, performance interfaces.

2.5.4 AI-Assisted Development (*Vibe Coding*)

Section 2.4 introduced *Vibe Coding* tools such as Lovable, Bolt.new and Figma Make, which influence the development workflow but are not part of the runtime architecture. In this project these tools are used to bootstrap React/Supabase prototypes and to generate much of the initial UI and CRUD logic for core workflows, or to convert high-fidelity Figma designs into React components that can be integrated into the main code base

These tools are not part of the runtime architecture of the platform, but they strongly influence the development workflow.

Generated code is treated as a starting point: it is refactored into a layered architecture and combined with a Spring Boot + PostgreSQL backend where critical concerns such as the data model, security, seriation logic, and auditing are implemented and reviewed. In this way, *Vibe Coding* accelerates time-to-prototype and exploration of alternatives, while the final institutional platform converges on a conventional, maintainable stack.

3

Requirements and Current System Analysis

3.1 Analysis of the Legacy System

The current internship management platform at DI at NOVA FCT is a web-based solution built on a deprecated technology stack. Although it supports the undergraduate internship life cycle, it presents critical limitations regarding security, architecture, and functionality. The section provides a systematic analysis of the current system's components and deficiencies, informed by direct exploration of the platform.

3.1.1 Infrastructure and Tech Debt

The system is hosted on a virtual machine and operates on an obsolete version of PHP dating back to approximately 2010, but since the platform was made even before that maybe there are some modules that are way older. This legacy codebase entails high maintenance risks and incompatibilities with modern libraries. Specifically, older PHP versions no longer receive security updates, leaving the platform vulnerable to known exploits that malicious actors routinely target [11].

A more critical vulnerability lies in the platform's network architecture: the system communicates exclusively over HTTP (without SSL/TLS encryption), representing a severe risk for transmitting sensitive student data and access credentials. Data transmitted over unencrypted HTTP is exposed to interception and eavesdropping, allowing attackers to read login credentials, personal information, and institutional data in plaintext [19].

Email Deliverability Crisis: A particularly severe technical deficiency resides in the email infrastructure. Due to the lack of modern authentication and deliverability configurations, transactional emails sent by the platform - intended for registration notifications, deadline reminders, call for proposals - are systematically flagged as spam by email providers [10]. This breakdown in communication disrupts the workflow between the department, students, and companies, reducing the operational effectiveness of the platform regardless of other functionality.

3.1.2 Functionalities and Limitations per Role

Administration and Coordination: The administrator profile provides a global overview of entities and proposals, allowing actions such as bulk student imports via CSV and basic permission management. A "*superuser*" feature exists, allowing administrators to access any user account without credentials for troubleshooting purposes. However, a critical gap exists in audit logging: while the system logs normal and superuser's logins and logouts, it lacks comprehensive action logging that tracks what users do during their sessions. For compliance and accountability, the platform should record specific administrative actions such as proposal status changes, student data modifications, permissions adjustments, and bulk import operations.

Content management relies on outdated, inflexible in-browser WYSIWYG editors for maintaining FAQs, Regulations, and Edition-specific information. Mass communication is rigid: the system offers no configurable templates for different stages of the workflow, relying instead on a single standard message for inviting companies.

Proposal Management and the Master thesis Gap: The platform includes interfaces for companies or professors to submit internship proposals and Master's dissertation themes. However, these dissertation theme submission and management workflows are completely inoperative - they exist as "dead data" in the system, forcing the department to manage dissertation assignments, publication and monitoring via external tools, negating the value of having collected this data in the platform.

For undergraduate internships, the "Project Classification" workflow allows coordinators to accept, reject, or request clarifications by annotating proposals. However, interaction remains limited: coordinators can toggle whether a proposal awaits company response, but the feedback mechanism lacks fluid integration for iterative proposal refinement.

Students and Companies: The student interface allows ranking of internship preferences and the consultation of imported academic records. For companies, human resources management is inefficient: there is no clear hierarchy or efficient mechanisms for managing multiple employees within a single company entity. The overall User Interface (UI) is outdated, hindering navigation and making it difficult for users to perceive current application status.

3.1.3 Data and Reports

The current system suffers from a critical lack of interoperability. There are no native features to export complex data (student lists, proposals, results of attributions) into open, standard format such as CSV or JSON, which complicates operations that the coordinators might need to make with the data outside the platform. Furthermore, company information is often outdated, as there are no periodic revalidation mechanisms to refresh company profiles and contact details ("Stale Company Data").

3.2 Requirements Gathering

Following the systematic analysis of the legacy platform and the identification of its critical failures, a comprehensive set of requirements was elicited for the new integrated platform. The elicitation process focused on addressing the needs of the diverse stakeholders involved in the academic lifecycle (students, companies, coordinators, secretariat, and administrators), with the goal of transitioning from fragmented spreadsheet-based management to a unified digital ecosystem supporting both undergraduate internships and master's dissertations. The resulting requirements were prioritized and refined in collaboration with the thesis advisers to ensure alignment with institutional constraints regarding data protection, interoperability, and operation feasibility.

3.2.1 Elicitation Methods

To ensure the new platform addresses real operational pain points and does not introduce new barriers to existing workflows, requirements were gathered through a multi-faceted approach:

- **Legacy System Analysis**: Direct exploration and reverse-engineering of the existing platform to identify functional gaps (e.g. non-functional Master's modules) and technical debt (e.g. email deliverability issues). This analysis, presented in Section 3.1, served as the foundation for identifying core functional requirements.

- **Document Analysis**: Review of current regulations, current internship rules, and manual processes currently used to manage Master's dissertations and feedback tracking. This provided insight into established business logic that must be preserved and systematized in the new platform.

- **Workflow Modeling**: Mapping of the "*as-is*" manual processes and system workflows against desired "*to-be*" automated and digitized workflow. Specifically for proposal submission and validation, student seriation algorithms, registrations workflows, company hierarchies, and feedback management. Process models were documented to clarify state transitions, decision points, and inter-stakeholder communication patterns.

- **Stakeholder Consultations**: Informal discussions with thesis advisers and the current coordinator responsible for the platform administration provided critical insights into administrative pain points, compliance requirements, and feature priorities. These discussions informed the prioritization of features and validated the proposed solution direction.

3.2.2 Stakeholders

The system must support multiple distinct user groups, with a role hierarchy that reflects the institutional structure in which Coordinators are Professors with administrative responsibilities for a given edition/s.

- **Students**: Consult internship and dissertation/project proposals for their edition, rank preferences on those proposals, track their candidacies and get assigned to one of them, access supervisor contacts, and, when required, submit supporting documents such as CVs or motivation letters.
- **Company Representatives**: Register and maintain company and department data, submit internship and Master's proposals during open calls for editions, respond to coordinator feedback, manage employee accounts linked to proposals, and monitor student candidacies for their proposals.
- **Professors**: Propose scientific dissertation themes for Master's students, supervise assigned students, and provide feedback on dissertation progress.
- **Edition Coordinators (Professor variant, Head Coordinator)**: Professors who configure and manage editions (type, deadlines, presentation slots, information pages), validate proposals through a feedback workflow, control attributions (including exceptional manual assignments), manage communication templates and bulk imports, and oversee protocols and audit logs, including permissions for sub-coordinators.
- **Sub-Coordinators (Professor variant)**: Professors delegated by the head coordinator to help validate and give feedback to proposals and manage edition-specific resources (FAQs, contact pages, email templates) within a given edition, without system-wide configuration permissions.
- **Secretariat/Administrative Staff**: Handle financial and legal formalization of internships and dissertations, including payment verification and protocol management, with read-only access to dashboards showing active internships and their status.

3.2.3 Functional Requirements

The functional requirements are categorized by feature set to ensure modular development, testing and clear traceability of stakeholder needs. Detailed specification for all functional requirements are presented in Annex I, Tables I.1-I.7. An overview of each category is provided here:

FR-1x - Edition and Configuration Management: Features for coordinators to create and configure editions (type, key dates) and maintain edition-specific content such as FAQs, regulations, contact information, and company presentation slots.

FR-2x - Proposal Submission and Classification: Workflows for companies and professors to submit and refine proposals through an iterative feedback process, including reuse of proposals across editions and optional presentation scheduling.

FR-3x - Student Candidacies and Seriation: Support for students to browse proposals, rank preferences, upload optional supporting documents, and for the system to run a seriation algorithm and notify students and companies about ranking and assignment results.

FR-4x - Company and User Management: Capabilities to validate companies and protocols, manage company structure and employee accounts, pre-register users in bulk, assign sub-coordinators, and offer controller *superuser* access with full audit logging.

FR-5x - Communication and Notifications: Automation of the "*Call for Proposals*", news and announcement publishing, and generation of email and in-app notifications for key events using configurable templates and a transactional email service.

FR-6x - Reporting and Data Export: Export of students, proposals, candidacies, and final assignments in open formats (CSV, JSON), plus detailed audit logs of critical actions and proposal feedback exchanges for accountability and compliance.

FR-7x - Master's Dissertation Lifecycle Support: Additional workflows for Master's dissertations and engineering projects, distinguishing between scientific and industry-oriented themes and tracking progress through milestones such as topic approval and final submission.

3.2.4 Non-Functional Requirements

To address the technical debt of the legacy system and ensure the platform meets institutional and user expectations, the new system must adhere to the following quality attributes, organized according to ISO/IEC 25010 standards. Detailed specifications for all-non functional requirements are presented in Annex I, Tables I.8-I.15. An overview of each category is provided here:

NFR-SEC-x - Security: Protects institutional and personal data via RLS, HTTPS/TLS encryption, secure password hashing, and role-based access control (RBAC) with granular permissions.

NFR-AUD-x - Auditability & Compliance: Ensures all critical actions (including proposal feedback and superuser sessions) are captured in tamper-evident audit logs with timestamps and user attribution, retained for compliance and dispute resolution [15].

NFR-REL-x - Reliability & Availability: Maintains availability during critical periods through a reliable transactional email service and replicated backups, and health monitoring with alerts for key components.

NFR-PERF-x - Performance & Scalability: Targets responsive page loads, and API latencies and peak load, supports real-time updates on key dashboards, and applies rate limiting to prevent abuse and protect shared resources.

NFR-USAB-x - Usability & Accessibility: Requires a responsive interface on desktop and mobile, clear visual feedback on interactions, and self-explanatory workflows (registration, proposal submission, ranking) supported by concise inline guidance.

NFR-INTER-x - Interoperability & Data Exchange: Provides bulk imports with validation and rollback, exports of key datasets in standard formats, and a documented REST API for secure, read-only integration with external institutional systems.

NFR-MAINT-x - Maintainability & Extensibility: Promotes a modular, layered architecture with externalized configuration, adequate documentation, and disciplined version control so that frontend, backend, and database can evolve independently.

NFR-EMAIL-x - Email Deliverability (Critical Legacy Fix): Requires authenticated outbound email, use of a professional transactional provider with monitoring and bounce handling, and tested templates to avoid spam classifications and rendering issues across common clients.

3.3 Use Cases

Use case modeling is a requirements engineering technique that describes interactions between actors (stakeholders) and the system to achieve specific goals. This section documents key use cases for the dissertation/internship management platform, representing critical workflows identified in the requirements' analysis. Each use case follows a standardized format including actors, preconditions, postconditions, main flow, and alternative flows.

3.3.1 Actors and Their Goals

Bases on the stakeholder analysis in Section 3.2.2, the platform involves the following actors:

Primary Actors (initiate use cases to achieve their own goals):

- *Student* - Seeks to discover proposals, rank preferences, and track candidacy status
- *Company Representative* - Aims to submit proposals and manage company participation
- *Professor* - Intends to propose dissertation themes and supervise students
- *Edition Coordinator* - Responsible for configuring editions and overseeing workflows

Secondary Actors (support the system but do not initiate the primary goal):

- *Secretariat* - Manage financial and legal documentation

- *System Administrator* - Maintains infrastructure and system health

3.3.2 High-Level Use Case Diagram

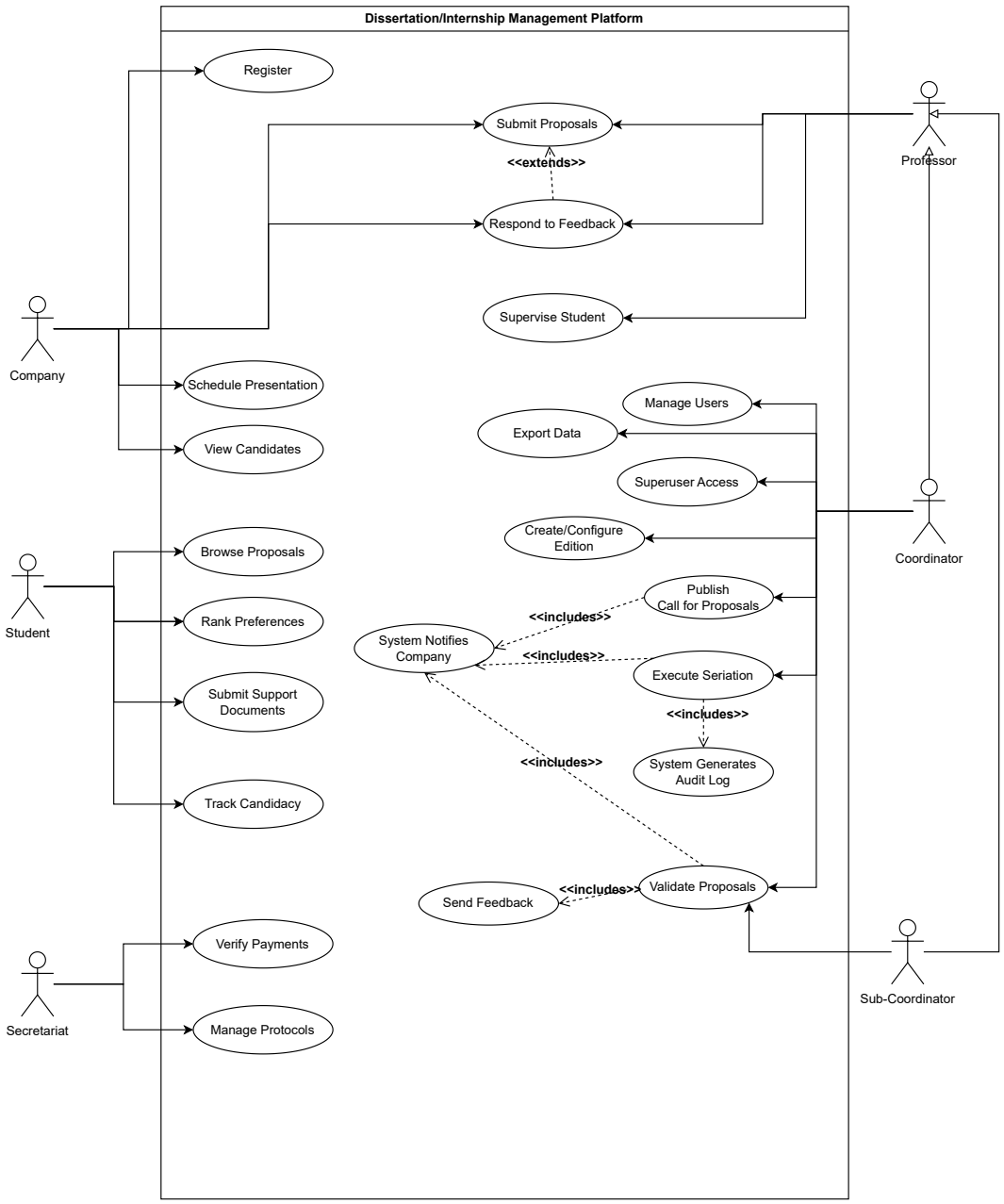

Figure 3.1: High-Level Use Case Diagram of the system with core functionalities

3.3.3 Detailed Use Case Specifications

This subsection documents the detailed specification of key use cases using the standard Cockburn template format: Use Case Name, Primary Actor(s), Secondary Actor(s), Preconditions, Postconditions, Main Flow, Alternative Flows [41]. The complete specifications for all use cases identified in this section are presented in Annex 2, Tables II.1-II.12.

The following describes the scope and organization of documented use cases:
Core Workflow Use Cases:

- **UC-01: Student Ranks Internship/Dissertation Preferences** - Central to the student candidacy phase; demonstrates the core functionality of the platform for students
- **UC-02: Coordinator Validates and Provides Feedback on Proposal** - Critical workflow; illustrates the iterative refinement loop between coordinator and company/professor
- **UC-03: System Executes Student Seriation Algorithm** - Core business logic; represents the automated ranking that differentiates this platform form manual processes
- **UC-04: Company/Professor Responds to Coordinator Feedback and Resubmits Proposal** - Demonstrates bidirectional communication; shows how feedback loops improve proposal quality
- **UC-05: Edition Coordinator Creates New Edition** - Administrative setup; establishes the foundation for almost all the other workflows
- **UC-06: Company Representatives Registers and Submits Initial Proposal** - Entry point for external stakeholders; illustrates onboarding and validation workflows.
- **UC-07: Student Submits Support Documentation** - Optional workflow, triggered when a coordinator requires CV or recommendation letters

Secondary Workflow Use Cases:

- **UC-08: Company/Professor Views Assigned Candidates List and Schedule Interviews** - Post seriation activities; enables company participation in selection
- **UC-09: Secretariat Verifies Payment and Finalizes Protocol** - Post-assignment; ensures legal and financial closure
- **UC-10: Edition Coordinator Publishes Call for Proposals** - Communication workflow; ensures all companies receive the invite to send proposals to the edition
- **UC-11: Administrator Performs Superuser Access for Student Support** - Administrative support; includes detailed audit logging requirements

Each use case specification includes:

- Actors - Primary (initiates the goal) and secondary (supports the goal) participants
- Preconditions - System state and permissions required before the use case can begin
- Postconditions - Guaranteed system state after the use case completes (success of failure)
- Main Flow - The primary, "happy path" sequence of steps
- Alternative Flows - Exception handling, edge cases, and variant paths (numbered A1, A2, A3, etc.) describing specific conditions where the main flow diverges.

Traceability to Requirements: Steps in the use case flows explicitly reference, when they have one, the corresponding Functional Requirement or Non-Functional Requirement (NFR-XX), ensuring complete traceability between user interactions and system capabilities.

3.3.4 Use Case Relationships and Extensions

The use case model also captures a small set of relationships that express shared or conditional behavior between use cases.

First, there is one extend relationship:

- Respond to Feedback extends Submit Proposals when the coordinator has requested clarifications. In this situation, the company follows an additional flow where it edits and resubmits the proposal instead of creating a completely new one.

Then, there are several include relationships, where one use case always performs another pas part of its normal execution:

- Validate Proposals includes Send Feedback, because every validation ends with the coordinator communicating an acceptance, rejection, or clarification request.
- Execute Seriation includes System Generate Audit Log, since each seriation run must be recorded with its parameters and results for auditability.
- Publish Call for Proposals, Execute Seriation, and Validate Proposals each include System Notifies Company, reflecting that companies are automatically informed when a new edition is open for proposals, when seriation results are available, or when their proposals change status.

3.3.5 Summary of Use Cases

4

Solution Proposal

4.1 Proposed System Architecture

The proposed platform follows a three-layer architecture that separates presentation, application logic and data persistence, while integrating technologies justified in Chapter 2.

At the presentation layer, a React/Next.js single-page application provides role-specific dashboards for students, company representatives, professors, edition coordinators, secretariat staff and system administrators. Next.js handles routing and page rendering (SSR/SSG), while React components implement reusable views such as proposal lists, candidacy tables and configuration forms.

The application layer is implemented as a set of backend services that expose a REST API consumed by the frontend. In the prototyping phase, these services are backed by Supabase (authentication, simple CRUD operations, real-time subscriptions), which allows rapid validation of core workflows. In the target architecture, a Spring Boot backend takes over the main business logic: proposal validation and feedback, student seriation, user and company management, protocol verification, notification orchestration and audit logging. Cross-cutting concerns such as authentication/authorization, email delivery and WebSocket-based real-time updates are handled centrally in this layer.

The data layer is built on PostgreSQL, using the relational schema that is described in Section 4.3. Core entities (Edition, User, Student, Company, Proposal, Candidacy, etc.) are linked through foreign keys to guarantee consistency, while RLS enforces fine-grained access control. JSON/JSONB columns are used for semi-structured data such as email templates and feedback metadata.

External services complete the architecture: a transactional email provider is responsible for reliable message delivery; object storage is used for uploaded documents (CVs, protocols, reports). This structure allows the initial AI-assisted prototypes to evolve into a maintainable, secure platform without discarding early work.

TODO: diagram maybe vai para os anexos

4.2 Process Modelling (BPMN)

TODO: mapping of workflows with bpmn

4.3 Data Model (ERD)

TODO: presentations of the ER diagram (db diagram) e definition of the entities there

4.4 Work Plan

TODO: maybe a cronogram (gantt?) with the development phases, tests, docs writing, risks and mitigations

Bibliography

[1] AlSalah. *Benefits and Drawbacks of Using Backend as a Service*. Point of SaaS. 2025. URL: https://pointofsaas.com/benefits-and-drawbacks-of-using-backend-as-a-service-3/ (cit. on p. 8).

[2] Anna Arteeva. *Choosing your AI prototyping stack: Lovable, v0, Bolt, Replit, Cursor, Magic Patterns compared*. 2025. URL: https://annaarteeva.medium.com/choosing-your-ai-prototyping-stack-lovable-v0-bolt-replit-cursor-magic-patterns-compared-9a5194f163e9 (cit. on p. 15).

[3] *App Router (Next.js)*. Next.js Documentation. Vercel. 2026. URL: https://nextjs.org/docs/app (cit. on p. 13).

[4] Ayush Sharma. *Difference between SQL and NoSQL*. GeeksforGeeks. 2025. URL: https://www.geeksforgeeks.org/sql/difference-between-sql-and-nosql/ (cit. on p. 10).

[5] B. Biskupski and A. Pytko-Włodarczyk. *Backend as a Service (BaaS) vs. custom backend: Which one to choose and why?* Accessed: 2026-01-17. 2019. URL: https://www.merixstudio.com/blog/backend-service-baas-vs-custom-backend (cit. on p. 8).

[6] *Chapter 15. Envers*. Hibernate Community. 2013. URL: https://docs.hibernate.org/orm/4.3/devguide/en-US/html/ch15.html (cit. on p. 8).

[7] P. P. Chen. "The Entity-Relationship Model: Toward a Unified View of Data". In: *ACM Transactions on Database Systems* 1.1 (1976), pp. 9–36 (cit. on p. 4).

[8] Cloudflare, Inc. *What is vibe coding?* 2025. URL: https://www.perplexity.ai/search/add-this-as-a-citation-in-late-dWWatfeETdCTpmIFNr6Jbg?sm=d (cit. on p. 14).

[9] W. contributors. *Vibe coding - chat based AI-assisted software development definition*. Accessed: 2026-01-17. 2026. URL: https://en.wikipedia.org/wiki/Vibe_coding (cit. on p. 4).

[10] David Clayton. *How to Improve Deliverability of Transactional Emails*. SMTP. 2023. URL: https://www.smtp.com/blog/transactional-email/improve-deliverability-of-transactional-emails/ (cit. on p. 19).

[11] Elena. *Why Using Legacy PHP Versions Makes Your Website Vulnerable*. Fastcomet. 2022. URL: https://www.fastcomet.com/blog/legacy-php-versions-makes-your-website-vulnerable (cit. on p. 19).

[12] *Embedded JavaScript templating Documentation*. EJS. 2026. URL: https://ejs.co/#docs (cit. on p. 9).

[13] *Express.js Documentation*. Express.js Foundation. 2026. URL: https://expressjs.com/ (cit. on p. 9).

[14] Figma. *What is Vibe Coding? Everything You Need to Know*. 2025. URL: https://www.figma.com/resource-library/what-is-vibe-coding/ (cit. on p. 7).

[15] Fortra. *Audit Log Best Practices for Security & Compliance*. 2024. URL: https://www.fortra.com/blog/audit-log-best-practices-security-compliance (cit. on p. 23).

[16] O. Glib. *Backend as a Service use Cases: How to apply*. Accessed: 2026-01-17. 2024. URL: https://acropolium.com/blog/baas-use-cases/ (cit. on p. 7).

[17] *Handlebars.js Documentation*. Handlebars.js. 2026. URL: https://handlebarsjs.com/ (cit. on p. 9).

[18] IBM Corporation. *Foreign key (referential) constraints*. IBM DB2 12.1.x Documentation. Accessed: 2026-01-21. 2026. URL: https://www.ibm.com/docs/en/db2/12.1.x?topic=constraints-foreign-key-referential (cit. on p. 10).

[19] kartik. *Difference between http:// and https://*. GeeksforGeeks. 2026. URL: https://www.geeksforgeeks.org/computer-networks/difference-between-http-and-https/ (cit. on p. 19).

[20] Mercy Tanga. *SSR vs. SSG in Next.js: Differences, Advantages, and Use Cases*. strapi. 2024. URL: https://strapi.io/blog/ssr-vs-ssg-in-nextjs-differences-advantages-and-use-cases (cit. on p. 13).

[21] Microsoft. *Joins (SQL Server)*. SQL Server Documentation. 2025. URL: https://learn.microsoft.com/en-us/sql/relational-databases/performance/joins?view=sql-server-ver17 (cit. on p. 10).

[22] Nafisa Anjum. *ReactJS Reconciliation*. GeeksforGeeks. 2025. URL: https://www.geeksforgeeks.org/reactjs/reactjs-reconciliation/ (cit. on p. 12).

[23] *Next.js Docs*. Vercel. 2026. URL: https://nextjs.org/docs (cit. on p. 13).

[24] *Node.js Documentation*. Node.js Foundation. 2026. URL: https://nodejs.org/en/docs/ (cit. on p. 9).

[25] Object Management Group. *Business Process Model And Notation (BPMN) Version 2.0*. OMG Document Number: formal/2011-01-03. Version 2.0. Object Management Group, 2011 (cit. on p. 4).

[26] Oskar Dudycz. *PostgreSQL JSONB - Powerful Storage for Semi-Structured Data*. 2025. URL: https://www.architecture-weekly.com/p/postgresql-jsonb-powerful-storage (cit. on p. 10).

[27] PostgreSQL Global Development Group. *8.14. JSON Types*. PostgreSQL 18 Documentation. 2026. URL: https://www.postgresql.org/docs/current/datatype-json.html (cit. on p. 10).

[28] Pranjal Nanda. *React Component Based Architecture*. GeeksforGeeks. 2025. URL: https://www.geeksforgeeks.org/reactjs/react-component-based-architecture/ (cit. on p. 12).

[29] K. Pykes. *What Are ACID Transactions? A Complete Guide for Beginners*. Datacamp. 2025. URL: https://www.datacamp.com/blog/acid-transactions (cit. on p. 10).

[30] React Team. *React*. Official React Documentation. Meta. 2025. URL: https://react.dev/ (cit. on p. 12).

[31] React Team. *React*. Meta Open Source Projects. Meta Open Source. 2026. URL: https://opensource.fb.com/projects/react/ (visited on 2026-01-23) (cit. on p. 12).

[32] J. Schibelli. *Vibe Coding: How AI is Transforming Software Development*. Accessed: 2026-01-17. 2025. URL: https://dev.to/johnschibelli/the-emergence-of-vibe-coding-how-ai-is-revolutionizing-software-development-4275 (cit. on pp. 7, 14).

[33] K. Schwaber and J. Sutherland. *The 2020 Scrum Guide*. Accessed: 2026-01-17. 2020. URL: https://scrumguides.org/ (cit. on p. 3).

[34] R. H. Shaikh. *Referential Integrity: Why It's Vital for Databases*. acceldata. 2024. URL: https://www.acceldata.io/blog/referential-integrity-why-its-vital-for-databases (cit. on p. 10).

[35] Spring Team. *Spring Boot Reference Documentation*. 3.x. Version 3.x. VMware, Inc. 2026. URL: https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/ (cit. on pp. 4, 8).

[36] Supabase. *Auth Overview*. 2026. URL: https://supabase.com/docs/guides/auth (cit. on p. 8).

[37] Supabase. *Edge Functions*. 2026. URL: https://supabase.com/docs/guides/functions (cit. on p. 8).

[38] Supabase. *REST API Overview*. 2026. URL: https://supabase.com/docs/guides/api#rest-api-overview (cit. on p. 8).

[39] Supabase. *Row Level Security*. 2026. URL: https://supabase.com/docs/guides/database/postgres/row-level-security (cit. on p. 8).

[40] Supabase. *Supabase: The Open Source Firebase Alternative*. https://github.com/supabase/supabase. Project README. 2019 (cit. on pp. 4, 8).

[41] Visual Paradigm. *A Comprehensive Guide to Use Case Modeling*. 2023. URL: https://guides.visual-paradigm.com/a-comprehensive-guide-to-use-case-modeling/ (cit. on p. 26).

[42] *What's the Difference Between an ACID and a BASE Database?* 2026. URL: https://aws.amazon.com/compare/the-difference-between-acid-and-base-database/ (cit. on p. 10).

[43] *What's the Difference Between Relational and Non-relational Databases?* 2026. URL: https://aws.amazon.com/compare/the-difference-between-relational-and-non-relational-databases/ (cit. on pp. 10, 11).

I

Annex 1 - Functional and Non-Functional Requirements

Functional Requirements

Table I.1: Functional Requirements - Edition and Configuration Management

ID	Description
FR-11	The system shall allow Edition Coordinators to create new academic editions, specifying edition type (Undergraduate Internship vs Master's Dissertation) and defining key timelines (proposal submissions deadline, interview periods, finalization deadline)
FR-12	The system shall allow Edition Coordinators to define and manage time slots for company proposal presentations, which companies can subsequently view and book during available windows
FR-13	The system shall enable Edition Coordinators to create and edit reusable email templates within the platform's interface, supporting dynamic variables (student names, company names, deadlines) and content-specific formatting for different communication stages.
FR-14	The system shall allow Edition Coordinators to create and maintain edition-specific informational pages (FAQs, Regulations, Contact Information, Important Dates) using a modern WYSIWYG editor.
FR-15	The system shall allow Edition Coordinators to publish news and announcements target at specific user roles (e.g., news visible only to Students, Companies, or Professors) within a given edition or platform-wide

Table I.2: Functional Requirements - Proposal Submission and Classification

ID	Description
FR-21	The system shall allow Companies and Professors to submit detailed proposals or dissertation themes, including attributes such as title, objectives, technical requirements, location, benefits, and supervisor information.
FR-22	The system shall implement a feedback loop allowing Edition Coordinators to request clarifications on specific aspects of proposals without full rejection, triggering email notifications to the author for editing and resubmission.
FR-23	The system shall allow Edition Coordinators to accept, conditionally accept, or reject proposals with detailed feedback comments.
FR-24	The system shall allow Companies to import or duplicate proposals from previous editions into the current edition, with the option to modify details before resubmission.
FR-25	The system shall allow Companies to indicate their intent to give oral presentations of their proposals and select available time slots directly within the platform.
FR-26	The system shall notify Companies of important deadlines (proposal submission closure, feedback response deadline, presentation scheduling deadline) via email and in-app notifications.
FR-27	The system shall notify Companies when a new edition opens in which they are eligible to submit proposals (Call for Proposals).

Table I.3: Functional Requirements - Student Candidacies and Seriation

ID	Description
FR-31	The system shall allow Students to view all available proposals for their enrolled edition and rank a specified number of preferred proposals in preference order.
FR-32	The system shall allow Students to submit supporting documentation (CV, motivation letter, recommendation letter) when registering interest in specific proposals, if required by the coordinator.
FR-33	The system shall execute an automated ranking algorithm based on predefine criteria (e.g. ECTS completed, grade average, coordinator-defined weights) to produce a seriation list of candidates for each proposal.
FR-34	The system shall notify Students of important timeline events via email and in-app notifications (registration opening, ranking deadline, seriation results, attribution confirmation)
FR-35	The system shall allow Companies to view the list of serialized (ranked) students for their proposals and manage candidate selections throughout the interview and confirmation phases.
FR-36	The system shall allow Companies to exclude or deselect candidates from their proposal's ranked list if needed.

Table I.4: Functional Requirements - Company and User Management

ID	Description
FR-41	The system shall allow Edition Coordinators to validate new company registrations, ensuring they sign the necessary legal protocols before submitting proposals
FR-42	The system shall allow Company Representatives to manage their organizational hierarchy, including parent company designation and sub-entities/departments, and to update company contact details and registration information
FR-43	The system shall allow Company Representatives to manage company employee accounts (add, remove, update) and designate which employees are responsible for specific proposals or serve as mentors.
FR-44	The system shall provide "Superuser" functionality, allowing Edition Coordinators to temporarily access other user accounts using only their user's identifier (student number or username) for support and troubleshooting purposes, with full audit logging of such access.
FR-45	The system shall allow Edition Coordinators to pre-register users (Students via CSV bulk import or manual entry, Professors, Secretariat staff) with role-based permissions.
FR-46	The system shall allow Edition Coordinators to assign or revoke the Sub-Coordinators role to Professors for specific reasons.

Table I.5: Functional Requirements - Communication and Notifications

ID	Description
FR-51	The system shall automate the "Call for Proposals" process, sending notifications to all registered companies (that have an active status) at the start of each edition, specifying submission deadlines and edition type.
FR-52	The system shall allow the publication of news and announcements targeted at specific user roles (e.g., news visible only to Students)
FR-53	The system shall enforce email deliverability best practices to minimize the likelihood of notifications being flagged as spam.
FR-54	The system shall support both email and in-app notifications for critical events (new proposals, feedback requests, deadline reminders, seriation results)
FR-55	The system shall allow Edition Coordinators to send targeted email communications to specific user groups (e.g., all enrolled Students, all registered Companies) with customizable subject and body content.

Table I.6: Functional Requirements - Reporting and Data Export

ID	Description
FR-61	The system shall allow Edition Coordinators and Administrators to export lists of students, proposals, candidacies, and final assignments in open standard formats (CSV, JSON).
FR-62	The system shall maintain detailed audit logs of critical system actions (user logins/logouts, proposal status changes, superuser access, permission modifications, bulk imports), with timestamps and user attribution.
FR-63	The system shall provide auditable records of all proposal feedback exchanges (coordinator feedback, company responses, timestamp of modifications) for compliance and dispute resolution.

Table I.7: Functional Requirements - Master's Dissertation Lifecycle Support

ID	Description
FR-71	The system shall support specific workflows for Master's degrees, including the distinction between scientific dissertations and engineering projects.
FR-72	The system shall allow Professors to propose scientific dissertation themes for Master's programs and manage their list of proposed themes and assigned students.
FR-73	The system shall enable tracking of dissertation progress and completing status, including milestones such as topic approval, and final submission/defense.
FR-74	The system should have edition for the Master's degree scientific dissertation and engineering projects that work the same way as the undergraduate internships, with just some changes.

Non-Functional Requirements

Table I.8: Non-Functional Requirements - Security

ID	Description
NFR-SEC-1	The system shall implement Row-Level Security at the database level to enforce strict data isolation, ensuring users can only access data they are authorized to view (e.g., companies see only their own proposals and candidates)
NFR-SEC-2	All data in transit shall be encrypted using HTTPS/TLS, preventing interception of sensitive information during transmission.
NFR-SEC-3	Password and sensitive credentials shall be hashed using industry-standard algorithms (bcrypt, PBKDF2, or Argon2) and shall never be stored in plaintext.
NFR-SEC-4	The system shall enforce role-based access control (RBAC) with granular permission assignment, ensuring users can only perform actions consistent with their assigned role.

Table I.9: Non-Functional Requirements - Auditability & Compliance

ID	Description
NFR-AUD-1	The system shall record an immutable, tamper-evident audit log of all critical actions (user logins/logouts, data modifications, status changes, permission assignments, superuser access) with precise timestamps and user attribution.
NFR-AUD-2	Audit logs shall be retained for long time to satisfy institutional compliance and legal hold requirements.
NFR-AUD-3	The system shall provide auditable records of all proposals feedback exchanges (coordinator feedback, company responses, timestamp of submission/modification) enabling dispute resolution and quality review.
NFR-AUD-4	Superuser access shall be logged in detail, including the identity of the accessing administrator, the target user account, timestamp, and duration of the session.

Table I.10: Non-Functional Requirements - Reliability & Availability

ID	Description
NFR-REL-1	The transaction email service shall guarantee a good deliverability rate to ensure time-sensitive notifications (e.g., proposals deadlines, seriation results) reach intended recipients reliably
NFR-REL-2	The system shall implement automated backups of all critical data (database, uploaded documents, configuration) with off-site replication to enable recovery in case of data loss.
NFR-REL-3	The system shall implement health monitoring and automated alerts for critical failures (database unavailability, backup failures, email service degradation), enabling rapid response by administrative staff.

Table I.11: Non-Functional Requirements - Performance & Scalability

ID	Description
NFR-PERF-1	All user-facing pages (dashboards, proposal lists, candidacy tracking) shall load within 3 seconds under normal load conditions.
NFR-PERF-2	Critical dashboards (Coordinator Edition Dashboard, Company Proposal Tracker, Student Status View) shall support real-time updates using WebSocket subscriptions or server-sent events, reflecting database changes without manual page refresh.
NFR-PERF-3	API endpoints shall respond within 500 ms for 95% of requests under typical load, and within 2 seconds for 99% of requests.
NFR-PERF-4	The system shall implement API rate limiting to protect against denial-of-service attacks and ensure fair resource allocation, limiting each user/API client to reasonable thresholds (e.g. 100 requests/minute for standard users, 300 requests/minute for bulk operations).

Table I.12: Non-Functional Requirements - Usability & Accessibility

ID	Description
NFR-USAB-1	The user interface shall be responsive and mobile-friendly, providing an optimal viewing experience on desktop (1920x1080) and mobile devices (320x568).
NFR-USAB-2	All interactive elements shall provide clear visual feedback (hover states, focus indicators, loading states) enabling users to understand the current application state.
NFR-USAB-3	New user workflows (registration, proposal submission, candidacy ranking) shall be completable without raining or external documentation, with clear inline guidance and error messages.

Table I.13: Non-Functional Requirements - Interoperability & Data Exchange

ID	Description
NFR-INTER-1	The system shall facilitate bulk import of users (Students, Professors, Secretariat) via CSV files with validation, error reporting, and rollback capabilities.
NFR-INTER-2	The system shall support export reporting data (student lists, proposals, seriation results, internship assignments) in standard formats (CSV, JSON) for use in external tools
NFR-INTER-3	The system shall provide a documented REST API for institutional integration, enabling third-party systems to query limited, read-only data.

Table I.14: Non-Functional Requirements - Maintainability & Extensibility

ID	Description
NFR-MAINT-1	The system architecture shall be modular and layered (presentation, business logic, data access), enabling independent updates to the frontend, backend, or database without cascading failures.
NFR-MAINT-2	The codebase shall be documented with README files, API documentation (Swagger/OpenAPI), and inline comments for complex business logic, enabling new developers to understand and modify the system.
NFR-MAINT-3	The system shall support semantic versioning and use Git-based version control with meaningful commit messages and branch-based workflows for traceability.
NFR-MAINT-4	Configuration (database URLs, email credentials, feature flags) shall be externalized using environment variables or configuration files, not hardcoded, to enable deployment across development, staging and production environments.

Table I.15: Non-Functional Requirements - Email Deliverability (Critical Legacy Fix)

ID	Description
NFR-EMAIL-1	All outbound emails shall implement sender authentication protocols to minimize spam flagging
NFR-EMAIL-2	The system shall integrate with a professional transaction email service with built-in deliverability monitoring and bounce handling.
NFR-EMAIL-3	Email templates shall be tested and validate before deployment to ensure correct rendering across major email clients (Gmail, Outlook, Apple Mail) and responsive display on mobile.

II

Annex 2 - Use Cases Specifications

Use Cases

Table II.1: Use Case Specification: Proposal Ranking

Use Case ID	UC-01
Use Case Name	Student Ranks Internship/Dissertation Preferences
Primary Actor	Student
Secondary Actors	Edition Coordinator, System
Preconditions	- Student is enrolled in current edition [FR-31]. - Proposal preference ranking period is open (not before start date, not after deadline). - At least one proposal is available for the edition.
Postconditions	- Student's preference ranking is saved in the system. - Confirmation message is displayed to the student. - Coordinator can retrieve data for seriation.
Main Flow	1. Student logs in and navigates to the edition they are enrolled in. 2. System displays all available proposals for the edition. 3. Student views details of each proposal (objectives, requirements, company, benefits, etc.). 4. Student selects and ranks a specified number of proposals in priority order. 5. System validates the number of proposals does not exceed the limit. 6. Student submits the ranking. 7. System confirms successful submission and displays a confirmation message. 8. System stores the ranking with timestamp [NFR-AUD-1].
Alternative Flows	**A1. Post-Deadline Attempt:** 1. System prevents submission; displays "Choosing period has closed". 2. Use case terminates. **A2. System Failure:** 1. System displays error, prompts user to try again. 2. Student's previous preference ranking remains unchanged (rollback).

Table II.2: Use Case Specification: Coordinator Validates and Provides Feedback on Proposal

Use Case ID	UC-02
Use Case Name	Coordinator Validates and Provides Feedback on Proposal
Primary Actor	Edition Coordinator
Secondary Actors	Company Representative/Professor, System, Email Service
Preconditions	- A new proposal has been submitted by a Company or Professor. - Edition Coordinator is assigned to this edition and has validation permission [FR-22]. - The proposal review period is active.
Postconditions	- Proposal feedback is recorded in the system. - Submitter is notified via email and in-app notification. - Proposal status is updated (pending clarification, accepted, or rejected). - Audit log records the coordinator's action with timestamp.
Main Flow	1. Edition Coordinator logs in and navigates to "Validate Proposals" page. 2. System displays list of submitted proposals awaiting validation. 3. Coordinator selects proposal to review. 4. System displays full proposal details with all submitted information. 5. Coordinator review proposal and identifies areas requiring clarification (or accepts/rejects). 6. If clarification needed: Coordinator writes specific feedback in the feedback form. 7. Coordinator selects "Request clarification" and submits [FR-22]. 8. System updates proposal status to "Awaiting Clarification". 9. System sends email to the submitter of the proposal with feedback details [FR-53]. 10. System posts in-app notification to the submitter's dashboard [FR-54]. 11. Audit log records: Coordinator name, action, feedback content, timestamp [FR-62].

Alternative Flows	**A1. Coordinator accepts proposal without changes:** 1. Step 6 is skipped. 2. Coordinator selects "Accept Proposal". 3. Proposal becomes visible to students for choosing. 4. Submitter is notified of approval via email and in-app notification. **A2. Coordinator rejects proposal:** 1. Coordinator selects "Reject Proposal" and provides a rejection reason 2. System updates status to "Rejected". 3. Submitter is notified with rejection reason via email [FR-53]. 4. Proposal is not visible to students **A3. Submitter fails to respond to clarification request within deadline:** 1. System sends automated reminder email 2 days before deadline. 2. After deadline, proposal automatically moves to "Deadline Exceeded" status 3. Coordinator can re-request clarification or reject.

Table II.3: Use Case Specification: System Executes Student Seriation Algorithm

Use Case ID	UC-03
Use Case Name	System Executes Student Seriation Algorithm
Primary Actor	System
Secondary Actors	Edition Coordinator, Student, Company Representative/Professor
Preconditions	- The seriation period has begun (coordinator has triggered seriation) - All students have submitted their preference rankings [FR-31] - All accepted proposals are locked (no further modifications allowed) - Pre-defined seriation criteria have been configured by coordinator.
Postconditions	- Seriation algorithm completes successfully - Assignments lists are generated for each proposal [FR-35] - Students are notified of seriation results via email and in-app notification - Companies/Professors receive the list of students assigned for each of their proposals - Audit log records seriation execution details.

Main Flow	1. Edition Coordinator logs in and navigates to "Run Seriation" 2. System displays a summary: number of students, number of proposals, seriation parameters (ECTS weight: X%, Average Grade weight: Y%) 3. Coordinator reviews and confirms parameters 4. Coordinator clicks "Execute Seriation" 5. System executes the seriation algorithm [FR-33]: • For each proposal, retrieves all student preference rankings where this proposal is ranked • Calculates a composite score for each student: (ECTSweight x studentECTS + AVGGradeweight x studentAVGGrade) • Sorts students by composite score in descending order • Produces a ranked list of candidates for each proposal 6. System generates proposal-to-candidate assignment results 7. System sends email notifications to all students listing their seriation results [FR-34] 8. System sends email notifications to all companies and professors with their ranked candidate lists [FR-35] 9. System displays "Seriation Complete" confirmation to coordinator 10. Audit log records: seriation execution time, number of students processed, algorithm parameters used [FR-62]
Alternative Flows	**A1. Algorithm detects data inconsistency (e.g., student has invalid Average Grade)** 1. System logs error and notifies coordinator 2. Seriation halts; coordinator must resolve data issue and retry **A2. No students ranked a particular proposal** 1. System marks that proposal with "No Candidates" status 2. Proposal is excluded from seriation results

Table II.4: Use Case Specification: Company/Professor Responds to Coordinator Feedback and Resubmits Proposal

Use Case ID	UC-04
Use Case Name	Company/Professor Responds to Coordinator Feedback and Resubmits Proposal
Primary Actor	Company Representative/Professor
Secondary Actors	Edition Coordinator, System
Preconditions	- Coordinator has requested clarification on company's/professor's proposal [UC-02, A3]. - Company/Professor has not yet exceeded the response deadline. - Company Representative/Professor is logged in.
Postconditions	- Modified proposal is resubmitted. - Coordinator is notified of resubmission. - Proposal is returned to "Under Review" status. - Feedback exchange is recorded in audit trail [FR-63].
Main Flow	1. Company Representative/Professor receives email notification of requested clarification. 2. Representative/Professor logs in to the platform and navigates to "My Proposals". 3. System displays the proposal marked "Awaiting Clarification" with coordinator's feedback highlighted. 4. Representative/Professor reviews feedback and modifies proposal details (e.g., clarifies location, adds requirements). 5. Representative/Professor adds a response comment addressing each point of feedback. 6. Representative/Professor clicks "Resubmit Proposal". 7. System validates modified proposal for completeness. 8. System updates proposal status to "Under Review". 9. System sends email to Edition Coordinator with notification of resubmission [FR-54]. 10. Audit log records: resubmission timestamp, modification details, company/professor response comment [FR-63].

Alternative Flows	**A1. Company/Professor does not respond before the deadline:** 1. System sends reminder email 2 days before deadline. 2. After deadline passes, proposal is automatically marked "Deadline Exceeded". 3. Coordinator can choose to reject or grant extension. **A2. Company's/Professor's modifications do not resolve the coordinator's concerns:** 1. Coordinator reviews resubmitted proposal (follows UC-02 main flow). 2. If still inadequate, coordinator can request additional clarification (cycle repeats).

Table II.5: Use Case Specification: Edition Coordinator Creates New Edition

Use Case ID	UC-05
Use Case Name	Edition Coordinator Creates New Edition
Primary Actor	Edition Coordinator
Secondary Actors	System
Preconditions	- Coordinator is authenticated and authorized as edition coordinator. - At least one academic program and default configuration template exist.
Postconditions	- A new edition record is created with status “Draft” or “Configured”. - Key parameters (dates, proposal limits, seriation rules) are stored. - Edition is available for later activation and publication.
Main Flow	1. Coordinator accesses the “Manage Editions” area. 2. System displays a list of existing editions and an option to create a new one. 3. Coordinator selects “Create Edition”. 4. System presents a form to define edition metadata (name, academic year, degree type, description). 5. Coordinator fills in dates (proposal submission window, student ranking window, seriation execution window). 6. Coordinator defines proposal constraints (max proposals per company, per professor, per student). 7. Coordinator configures seriation parameters (e.g., average grade and ECTS weights, tie-breaking rules). 8. Coordinator saves the edition. 9. System validates the data, creates the edition in “Draft/Configured” status and confirms creation.

Alternative Flows	**A1. Invalid or missing data:** 1. At step 9, validation fails (e.g., end date before start date). 2. System shows specific validation errors and highlights problematic fields. 3. Coordinator corrects the data and retries saving. **A2. Duplicate edition:** 1. System detects an existing edition with conflicting key identifiers (e.g., same academic year and cycle). 2. System warns coordinator and suggests editing the existing edition instead. 3. Coordinator either cancels or changes identifiers and saves again.

Table II.6: Use Case Specification: Company Representative Registers and Submits Initial Proposal

Use Case ID	UC-06
Use Case Name	Company Representative Registers and Submits Initial Proposal
Primary Actor	Company Representative
Secondary Actors	System, Secretariat (for protocol validation), Coordinator
Preconditions	- Call for proposals for at least one edition is open. - Representative has a valid email address.
Postconditions	- A new company account (or user under an existing company) is created and marked as “Pending Validation” or “Active”. - At least one proposal is stored with status “Submitted” or “Pending Validation”. - Coordinator can see the proposal in the validation queue.
Main Flow	1. Representative accesses the platform and chooses “Register Company / Representative”. 2. System displays a registration form (company info, contact details, legal identifiers). 3. Representative fills in the form and submits. 4. System validates data, creates the company (or links user to existing company), and sends an activation email. 5. Representative activates the account via the email link and logs in. 6. Representative navigates to “Submit Proposals”. 7. System shows available editions and their submission windows. 8. Representative selects an edition and fills in proposal data (title, description, objectives, requirements, location, compensation/benefits). 9. Representative submits the proposal. 10. System validates the proposal, stores it with status “Submitted/Pending Validation”, and shows a confirmation.

Alternative Flows	**A1. Company already exists:** 1. At step 3, system detects that the company exists. 2. System offers to link the new user to the existing company instead of creating a new one. 3. Representative confirms and proceeds; account is created under that company. **A2. Registration incomplete:** 1. Representative closes the browser or cancels before activation. 2. System keeps a temporary record, which can be completed later via emailed link or is purged after a timeout. **A3. Proposal submitted after deadline:** 1. At step 8, if the edition's proposal window is closed, system blocks submission. 2. System shows an error and suggests choosing another open edition (if available).

Table II.7: Use Case Specification: Student Submits Support Documentation

Use Case ID	UC-07
Use Case Name	Student Submits Support Documentation
Primary Actor	Student
Secondary Actors	Coordinator, System
Preconditions	- Student is authenticated and enrolled in an edition. - Student has already created or is creating a ranking of proposals. - Coordinator or edition configuration requires supporting documents for that edition or specific proposals.
Postconditions	- Supporting documents (e.g., CV, motivation letter, recommendations) are uploaded and linked to the student's candidacies. - Coordinator and companies (where applicable) can access documents during seriation and evaluation.
Main Flow	1. Student opens the "My Candidacies / Rankings" page. 2. System indicates which candidacies or editions require supporting documents. 3. Student selects a candidacy requiring documents. 4. System displays required and optional document types (e.g., CV required, motivation letter optional). 5. Student uploads the requested files and confirms. 6. System validates file types and sizes, stores them securely, and links them to the corresponding candidacy. 7. System shows a confirmation and updates the candidacy status to "Documents Submitted".

Alternative Flows	**A1. Missing required document:** 1. At step 5, student omits a required file. 2. System highlights the missing document and prevents completion until it is provided. **A2. Invalid file:** 1. Uploaded file exceeds size limit or uses a forbidden type. 2. System rejects that file and informs the student of allowed types and limits. **A3. Update documents:** 1. Before the document deadline, student returns to the candidacy. 2. System allows replacing or adding documents. 3. New versions are stored and previous versions are either archived or overwritten.

Table II.8: Use Case Specification: Company/Professor Views Assigned Candidates List and Schedules Interviews

Use Case ID	UC-08
Use Case Name	Company/Professor Views Assigned Candidates List and Schedules Interviews
Primary Actor	Company Representative/Professors
Secondary Actors	Coordinator, System
Preconditions	- Seriation for the relevant edition has been executed successfully. - Company/Professor has at least one accepted proposal in that edition. - Representative/Professor is authenticated and linked to that company.
Postconditions	- Representative/Professor can see ranked or assigned candidates per proposal.
Main Flow	1. Representative/Professor logs in and navigates to "My Proposals / Candidates". 2. System lists the company's/professor's proposals for the edition and their current seriation status. 3. Representative/Professor selects a proposal. 4. System displays the ranked candidate list or assigned students, including basic profile data allowed by policy. 5. Representative/Professor optionally selects one or more candidates get their contacts to schedule interview outside the platform.
Alternative Flows	**A1. No candidates available:** 1. At step 4, system indicates that no candidates were ranked for that proposal. 2. Representative/Professor may close the view or contact the coordinator outside this use case.

Table II.9: Use Case Specification: Secretariat Verifies Payment and Finalizes Protocol

Use Case ID	UC-09
Use Case Name	Secretariat Verifies Payment and Finalizes Protocol
Primary Actor	Secretariat
Secondary Actors	Company Representative, Student, System
Preconditions	- A student has been assigned to a proposal that requires a protocol or payment. - Required protocol documents and/or payment information are available (uploaded or received externally). - Secretariat staff is authenticated with appropriate permissions.
Postconditions	- Payment status and protocol status are updated (e.g., "Verified", "Signed"). - The associated internship/dissertation is marked as "Active" or "Authorized". - Relevant parties can see the updated status.
Main Flow	1. Secretariat logs in and opens the "Protocols / Payments" section. 2. System shows a list of pending cases (assigned proposals requiring verification). 3. Secretariat selects a specific case. 4. System shows student, company, proposal details, and any uploaded documents or payment records. 5. Secretariat verifies that all required documents are signed and any required payments are confirmed. 6. Secretariat updates the status to "Verified / Finalized" and saves. 7. System updates the associated assignment to "Active" and notifies the company and student.

Alternative Flows	**A1. Missing or invalid documentation:** 1. At step 5, Secretariat detects missing signatures or incorrect information. 2. Secretariat sets the status to "Incomplete" and records a comment. 3. System notifies the company and/or student with the requested corrections. **A2. Payment not received:** 1. Payment is not yet confirmed. 2. Secretariat records the issue and leaves the status as "Pending Payment". 3. No activation occurs until payment is confirmed.

Table II.10: Use Case Specification: Edition Coordinator Publishes Call for Proposals

Use Case ID	UC-10
Use Case Name	Edition Coordinator Publishes Call for Proposals
Primary Actor	Edition Coordinator
Secondary Actors	System, Company Representatives
Preconditions	- At least one edition exists in "Configured" state with defined proposal submission dates. - Coordinator is authenticated and has edition management rights. - Company contacts exist or can be targeted by the call.
Postconditions	- The edition's call for proposals is marked as "Published". - Companies can see the call when logging in and can submit proposals for that edition. - Notification emails are sent to targeted companies.
Main Flow	1. Coordinator opens the "Editions" view and selects a configured edition. 2. System shows current configuration and call status. 3. Coordinator reviews details (dates, constraints) and chooses "Publish Call for Proposals". 4. System requests confirmation and optionally allows editing the email template. 5. Coordinator confirms publication. 6. System changes the edition state to "Call Published / Open for Proposals". 7. System sends notification emails to configured company recipients and updates the dashboard.
Alternative Flows	**A1. Misconfigured edition:** 1. At step 3, system detects missing critical parameters (e.g., no submission window). 2. System blocks publication and presents a list of missing items. 3. Coordinator fixes the configuration, then retries. **A2. Email sending issues:** 1. At step 7, some emails fail to send. 2. System logs failures and informs the coordinator, offering a way to export the list of failed addresses.

Table II.11: Use Case Specification: Administrator Performs Superuser Access for Student Support

Use Case ID	UC-12
Use Case Name	Administrator Performs Superuser Access for Student Support
Primary Actor	System Administrator
Secondary Actors	Student, Coordinator, Company Representative, System
Preconditions	- Administrator is authenticated with elevated permissions. - A support request has been raised (e.g., user cannot access their account, data inconsistency, wrong role assignment). - Superuser actions are enabled and auditable.
Postconditions	- The specific problem (access, data correction, role fix) is resolved. - All actions executed under superuser mode are logged with timestamp, actor, and justification.
Main Flow	1. Administrator opens the "Support / Superuser Access" area. 2. System lists open support tickets or allows searching by user, edition, or proposal. 3. Administrator selects a case and views context (user roles, assignments, logs). 4. Based on the case the administrator logs in the Superuser with the other user's login. 5. Administrators troubleshoots the problems. 6. Logs out of the accounts being accessed. [A2]
Alternative Flows	**A1. Action not allowed by policy:** 1. The requested operation violates locked-data policy. 2. System blocks the action and explains that such changes require a different process. 3. Administrator records this outcome in the ticket. **A2. Impersonation session ended:** 1. During impersonation, administrator finishes troubleshooting. 2. Administrator exits superuser session explicitly. 3. System returns to the administrator's own account and logs the full session.